Crop Percentage Detector — Task Brief

What we already have at PinIt

We already have two different systems that deal with cropping, for context:

1. **Proving an image is still yours, even after heavy cropping.** When we protect an image, we hide a repeating watermark across the whole image (many small tiles, not one signature in a corner). As long as roughly 20%+ of the original survives in the crop, enough tiles usually remain to recover "this belongs to this owner."
2. **Comparing two images to see if one is a cropped copy of the other.** This runs during an investigation — someone uploads a suspected copy, and the system checks it against the vault using feature matching and geometry (explained below).

Your task is a *new, standalone* version of problem #2, but with a sharper goal: not just "is this a crop," but **exactly what percentage was cropped**, accurately from light crops (10%) all the way to heavy ones (90%).

The task, in one sentence

Given an **original image** and a **cropped version** of it, calculate **what percentage of the original was cut away** — accurately whether the crop is small (10%) or very large (90%), and ideally down to the pixel.

Think of it like this: if someone hands you a full newspaper page and then a smaller clipping cut from somewhere on that page, can you figure out exactly which rectangle was clipped, and therefore how much of the page is *not* in your hand? That's the whole task.

The core idea

If you can figure out **where** the cropped image sits inside the original, and **how big** that region is, the rest is just area math:

```
\text{crop\%} = 100 \times \left(1 - \frac{\text{area of the matched region}}{\text{area of the original image}}\right)
```

So the real problem isn't the percentage math — it's finding that matched region accurately. That's what the two examples below teach, from easiest to hardest.

Example 1 — the easy starting point: exact crop (template matching)

Start here, even though it's not the real-world case. It teaches the fundamentals.

Setup: the cropped image is just a rectangle cut directly out of the original — no resizing, no rotation, same pixel scale.

Real-world analogy: imagine a cardboard stencil cut to the exact shape of part of a photo. If you slide that stencil around over the photo, there is exactly one position where it lines up perfectly — pixel for pixel.

How to do it: this is a solved, classic computer-vision operation called **template matching**. You slide the small (cropped) image over the big (original) image, position by position, and measure how different the pixels are at each position. The position with the smallest difference is where the crop came from.

- In OpenCV (Python): `cv2.matchTemplate()`.
- It gives you back the exact (x, y) position and size of the matched rectangle.

Worked example:

- Original photo: 1000 × 1000 px = 1,000,000 px²
- Cropped piece found at some position, size 600 × 400 px = 240,000 px²
- crop% = 100 × (1 − 240,000 / 1,000,000) = **76% cropped**

Build this first. It's a great warm-up and it will work perfectly as long as the crop wasn't also resized or compressed afterward — which it usually is in real life (see Example 2).

Example 2 — the real case: crop + resize + compression

In real life, people don't just crop — they crop *and then resize* (e.g. crop to a square and re-export at 1080×1080 for Instagram), and the file usually gets re-compressed too. Now the cropped image's pixel dimensions tell you nothing directly about the crop percentage, and template matching from Example 1 won't line up anymore.

Real-world analogy: imagine the full word **PINEAPPLE**, and someone shows you just **APPLE** — but in a different font size and a bit blurry. You can't measure letter widths and compare directly. Instead you recognize the *shape and pattern* of the letters themselves, figure out which letters they are, and work out where "APPLE" sits inside "PINEAPPLE" and how much it was stretched or shrunk.

How to do it — feature matching + homography:

1. **Find distinctive points** in both images — corners, edges, unique little textures (a doorknob, a leaf vein, an eye corner). In OpenCV: `cv2.ORB_create()` (fast, free) or `cv2.SIFT_create()` / `AKAZE` (slower, sometimes more accurate).
2. **Match those points** between the two images — which point in the crop corresponds to which point in the original. OpenCV: `cv2.BFMatcher` or `cv2.FlannBasedMatcher`.
3. **Work out the transform** that explains how the matched points relate — this is called a **homography**: a formula for "shift this much, scale this much, maybe rotate this much." OpenCV: `cv2.findHomography(..., method=cv2.RANSAC)`.

Why RANSAC matters (explained simply): some of your point matches from step 2 will be *wrong* — a texture that coincidentally looks similar in two unrelated spots. RANSAC is a clever trick: it repeatedly tries small random groups of matches, checks which group's resulting transform best explains *all the other* matches too, and throws away the ones that don't fit. Think of it like fitting a straight line through a scatter of dots where most dots roughly follow a trend but a few are noise — RANSAC finds the best-fitting line while ignoring the noisy dots.

Once you have that transform, you know exactly how the crop relates to the original — including any resizing — and you can calculate the percentage from there (see next section).

Getting pixel-level precision

Once you have the homography transform from Example 2, you can take the **four corners of the cropped image** and project them back onto the original image's coordinate system (`cv2.perspectiveTransform`). That gives you the exact rectangle, in original-image pixels, that the crop came from — not an estimate.

From there:

```
\text{kept area} = \text{width}_{\text{original px}} \times
\text{height}_{\text{original px}}
```

```
\text{crop\%} = 100 \times \left(1 - \frac{\text{kept area}}{\text{original width}
\times \text{original height}}\right)
```

This works even if the crop was at a slight angle or rotated — a homography is a general transform, not just "scale and shift."

Edge cases to think about and test

- **Very heavy crops (80–90%)** leave very few feature points to match — accuracy will likely drop here. Test this specifically; it may need a fallback (see below).
- **Rotated crops** should still work — homography handles rotation, not just straight rectangles — but test it to be sure.
- **Extra edits after cropping** (filters, text overlays, heavy compression) can confuse feature matching. ORB is reasonably robust to this, but verify.
- **Unrelated images** should NOT report a crop percentage — the tool should be able to say "this doesn't match the original" instead of forcing a number.
- **Fallback idea for very heavy crops:** if feature matching finds too few reliable points, fall back to comparing small patches/blocks of the original against the crop and "voting" on the best-matching area — similar in spirit to how PinIt's own system falls back to local-patch matching for heavy crops.

How to research this well — a concrete study path

Go in this order; each step builds on the last:

1. **OpenCV basics in Python** — loading images, drawing rectangles, cropping with Pillow/PIL (`image.crop()`). You'll need this to generate your own test crops too.
2. **Template matching** (`cv2.matchTemplate`) — build Example 1 above first. This teaches pixel comparison and bounding-box math before anything harder.
3. **Feature detectors** — learn ORB first (fast, free, good enough to start); read about SIFT and AKAZE as alternatives once ORB works.
4. **Feature matching + RANSAC + homography** — search specifically for the official OpenCV tutorial titled **"Feature Matching + Homography to find Objects"**; it is almost exactly this task.
5. **Image registration / panorama stitching tutorials** — these use the *exact same math* (figuring out how two overlapping photos relate to each other), so they transfer directly and there's a lot of good material written about them.

6. **Build your own test crops before writing the detector** — use Pillow's `.crop()` to cut known, exact percentages (10%, 20%, ... 90%) out of a handful of real photos. Keep a note of the *true* crop percentage for each one you generate — you'll need this for validation (next section).

The small tool to build

Keep it deliberately simple — this is a test/demo tool, not a product feature.

Flow:

1. **Upload original** — one button, one image. Save it (to a local folder is fine for now, no database needed).
2. **Upload a crop, every time you want to test** — one button, one image. It gets compared against the saved original and the page shows: *"This image is approximately X% cropped from the original."*

Nice-to-have (not required for v1): draw a box on the original image showing exactly which region was detected as "kept" — this is a great way to visually sanity-check whether the detection is actually right, not just trust the number.

Suggested stack: Python backend (Flask or FastAPI — OpenCV is strongest in Python) with two endpoints, `/upload-original` and `/compare-crop`, plus a bare-bones HTML page or a simple React page with two upload buttons. No auth, no database needed — this is a local testing tool, not something that goes to production.

How to validate it actually works

Don't just eyeball the results — build a known-answer test set first, the same way you'd grade a quiz you already have the answer key for:

1. Pick 5–10 varied real photos (different subjects, different amounts of detail/texture).
2. For each one, programmatically crop it at exact known percentages — 10%, 20%, 30% ... 90% — using code (Pillow's `.crop()`), never by eye. This gives you the *true* answer for every test image.
3. Run your detector on each cropped image and record what it predicts.
4. Compare predicted % vs. true % for every case, across every photo and every crop level.

This turns "it seems to work" into an actual accuracy report — e.g. "accurate within 2% up to a 70% crop, degrades beyond that." That's the kind of result that's actually useful to report back.